

Capítulo 16

Introdução a Classes e Objetos

Exercícios de Autorrevisão — Resolução Didática

“Eis o capítulo do salto conceitual: deixamos de pensar em funções que operam sobre dados para passar a pensar em classes — tipos que encapsulam, em uma só unidade, os dados (atributos) e o comportamento (funções-membro) das entidades do nosso domínio. É o passo decisivo da programação procedural para a programação orientada a objetos.”

1 Exercício 16.1 — Vocabulário de classes

Preencha os espaços em cada uma das sentenças (a)–(j).

(a) Uma casa está para uma planta como

está para uma classe

Resposta: um objeto.

A analogia é elucidativa: assim como muitas casas podem ser construídas a partir de uma única planta arquitetônica, muitos *objetos* podem ser instanciados a partir de uma única definição de *classe*. A planta é *prescritiva* (descreve o que toda casa daquele tipo deverá ter); cada casa é *concreta* (tem suas próprias dimensões finais, sua localização, seus moradores). Do mesmo modo, a classe **Conta** estipula que toda conta tem um saldo e suporta depósitos e saques; cada objeto **Conta** concreto possui o seu próprio saldo, sua própria história.

Observação de Engenharia de Software

A separação classe-versus-objeto é tão fundamental que merece destaque visível. Em UML, classes são retângulos com nome em negrito; objetos são retângulos com nome sublinhado e prefixado pelo nome do objeto: **conta1** : **Conta**. É ao confundir os dois níveis que o iniciante mais frequentemente se atrapalha.

(b) Toda definição de classe contém a palavra-chave

Resposta: `class`.

A sintaxe canônica é:

```
class NomeDaClasse {  
public:  
    // funcoes-membro publicas (interface)
```

```
private:
    // dados-membro privados (implementacao)
};    // nao esqueca o ponto e virgula!
```

Erro Comum de Programação

Esquecer o ponto-e-vírgula final após a chave de fechamento de uma `class` é um dos erros mais frequentes dos iniciantes em C++. Os erros de compilação decorrentes são crípticos e geralmente apontam para a *próxima* declaração — não para a classe em si. Lembre-se: `class`, `struct` e `union`, todas terminam em `};`.

(c) Extensão do arquivo que contém a definição da classe

Resposta: `.h`.

A convenção em C++ é armazenar a *declaração* da classe em um **arquivo de cabeçalho** (*header*) com extensão `.h` (ou, às vezes, `.hpp`) e a *implementação* de suas funções-membro em um **arquivo de código-fonte** com extensão `.cpp`. Veremos essa divisão em ação nos exercícios 16.11 a 16.17, onde cada classe terá o seu par `.h/.cpp`.

(d) Cada parâmetro deve especificar

Resposta: um **tipo** e um **nome**.

Em `void f(int x, double y)`, cada parâmetro tem dois componentes obrigatórios: o tipo (`int`, `double`) e o nome (`x`, `y`). O tipo informa ao compilador como interpretar os bits recebidos; o nome é usado dentro do corpo da função para referir-se ao valor.

(e) Quando cada objeto mantém sua própria cópia de um atributo, este é chamado

Resposta: um **dado-membro** (ou simplesmente *membro de dado*).

Os dados-membro são as variáveis declaradas *dentro* da definição da classe (mas *fora* de qualquer função-membro). Cada objeto da classe carrega sua própria cópia de cada dado-membro. Por isso, dois objetos da mesma classe podem ter estados *distintos*: `conta1` pode ter saldo 100 enquanto `conta2` tem saldo 500.

Observação de Engenharia de Software

Há uma exceção a essa regra: se um membro for declarado `static`, ele é *compartilhado* por todos os objetos da classe — existe uma só cópia. Membros estáticos são tema do Capítulo 17.

(f) A palavra-chave `public` é um**Resposta:** especificador de acesso.

Os três especificadores de acesso em C++ são `public`, `private` e `protected`. Eles controlam quem pode ver e usar cada membro:

- `public`: acessível de *qualquer lugar* (clientes da classe).
- `private`: acessível *apenas* a partir das funções-membro da própria classe.
- `protected`: como `private`, mas também acessível a classes derivadas (herança; Cap. 18).

(g) Tipo de retorno que indica que a função não devolve informação**Resposta:** `void`.

Quando uma função executa uma tarefa mas não tem nada específico a retornar (por exemplo, exibir uma mensagem na tela, ou modificar um dado-membro), declaramo-la com retorno `void`. É o mesmo `void` que já usávamos em C; nada muda aqui.

(h) Função de `<string>` que lê até encontrar *newline***Resposta:** `getline`.

O operador `>>` de `cin` lê até o primeiro espaço em branco. Para ler uma linha inteira (que pode conter espaços), usa-se `getline(cin, minhaString)`, que lê até o caractere de nova linha (`\n`). É o análogo C++ do `fgets` de C.

(i) Operador que “amarra” a função-membro à classe**Resposta:** operador binário de resolução de escopo (`::`).

Lembre-se do Capítulo 15: o `::` unário acessa o escopo global. Aqui usamos a forma *binária*: `NomeDaClasse::nomeDaFuncao` significa “a função `nomeDaFuncao` *pertencente* à classe `NomeDaClasse`”. É indispensável quando definimos fora da classe:

```
// No arquivo .h:
class Conta {
public: int getSaldo() const;
private: int saldo;
};

// No arquivo .cpp:
int Conta::getSaldo() const { return saldo; }
// ~~~~~ <-- operador binario de resolucao de escopo
```

(j) Diretiva de pré-processador para incluir o cabeçalho**Resposta:** `#include`.

A mesma diretiva já familiar de C. A diferença notável: cabeçalhos da biblioteca-padrão em C++ são escritos sem extensão — `<iostream>`, `<string>`, `<vector>` — enquanto cabeçalhos do programador continuam com extensão `.h` e entre aspas: `#include "MinhaClasse.h"`.

2 Exercício 16.2 — Verdadeiro ou falso

(a) Nomes de função começam com maiúscula

Resposta: Falso.

A convenção adotada pelos autores deste livro é *camel case* (com primeira letra minúscula): a primeira palavra começa minúscula, e as palavras subsequentes começam maiúsculas. Exemplos: `displayMessage`, `getCourseName`, `obterIdade`. Por outro lado, os *nomes de classes começam* com maiúscula (`GradeBook`, `Account`, `Date`) — convenção chamada *PascalCase*.

(b) Parênteses vazios indicam que a função não tem parâmetros

Resposta: Verdadeiro.

`void f();` declara uma função sem parâmetros. Em C++, a lista vazia significa o que parece; em C antigo (não em C moderno), uma lista vazia significava “número não especificado de parâmetros”.

(c) Membros `private` são acessíveis às funções-membro da classe

Resposta: Verdadeiro.

A regra do `private` é: *fora* da classe, não se vê; *dentro* da classe (em qualquer função-membro), vê-se livremente. É essa assimetria que viabiliza o *encapsulamento*: os clientes da classe enxergam apenas a interface pública; os detalhes ficam isolados, podendo ser alterados sem afetar o código cliente.

(d) Variáveis no corpo de uma função-membro são dados-membro

Resposta: Falso.

Variáveis declaradas no *corpo* de uma função-membro são **variáveis locais**: existem somente durante a execução daquela função e são visíveis apenas dentro dela. Para serem dados-membro, devem ser declaradas no nível da *classe*, fora de qualquer função-membro.

(e) O corpo de toda função é delimitado por chaves

Resposta: Verdadeiro.

`{` abre, `}` fecha. É uma regra rígida em C++. (As linguagens que dispensam chaves, como Python, usam indentação significativa; não é o caso aqui.)

(f) Qualquer arquivo com `int main()` pode executar como programa**Resposta: Verdadeiro.**

`main` é o *ponto de entrada* de todo programa C++. Por definição, deve existir exatamente uma função com esse nome em todo o programa, e é ela quem o sistema operacional invocará primeiro.

(g) Tipos dos argumentos devem ser coerentes com os parâmetros**Resposta: Verdadeiro.**

Quando você chama `f(3.14)` e `f` foi declarada como `int f(int)`, o compilador realizará uma *conversão implícita* (com perda de informação) de `double` para `int`. Para tipos incompatíveis (string para `int`, por exemplo), o compilador rejeitará a chamada.

3 Exercício 16.3 — Variável local versus dado-membro

Qual é a diferença entre uma variável local e um dado-membro?

Resposta didática expandida

Aspecto	Variável local	Dado-membro
Onde é declarada	No corpo de uma função	Dentro da classe, fora das funções-membro
Quanto tempo vive	Da declaração até o <code>}</code> do bloco	Enquanto o objeto existir
Quem vê	Só a própria função	Todas as funções-membro do objeto
Por objeto	Não se aplica	Cada objeto tem sua cópia
Inicialização	Lixo se não inicializada	Tratada pelo construtor

Exemplo concreto

```

1 class Conta {
2 public:
3     void depositar(int valor) {
4         int novoSaldo = saldo + valor; // <-- VARIÁVEL LOCAL
5         saldo = novoSaldo;
6     }
7 private:
8     int saldo; // <-- DADO-MEMBRO
9 };

```

Na linha 3, `novoSaldo` é uma variável local: nasce no `int novoSaldo = ...`, morre no `}` da função, não é vista por outras funções. Na linha 7, `saldo` é um dado-membro: vive enquanto o objeto `Conta` existir, e é visível a todas as funções-membro.

Boa Prática de Programação

Quando uma variável local tem nome igual ao de um dado-membro, a local *oculta* o dado-membro — e isso é fonte de bugs sutis. Adote uma convenção de nomenclatura que evite a colisão: alguns programadores usam um sufixo ‘_’ nos dados-membro (`saldo_`), outros prefixam com `m_` (`m_saldo`). Os autores deste livro evitam esses adornos e simplesmente cuidam para não reusar nomes.

4 Exercício 16.4 — Parâmetro versus argumento

Explique a finalidade de um parâmetro de função. Qual é a diferença entre parâmetro e argumento?

Resposta didática expandida

Parâmetro é o nome que aparece na *declaração* ou *definição* da função — é uma variável especial que receberá o valor passado pelo chamador. **Argumento** é o valor concreto fornecido na *chamada* da função.

Exemplo

```

1 // PARAMETROS: count, radius (no cabeçalho)
2 double areaCirculo(double radius) {
3     return 3.14159 * radius * radius;
4 }
5
6 int main() {
7     double r = 5.0;
8     double a = areaCirculo(r);           // ARGUMENTO: r (valor 5.0)
9     double b = areaCirculo(10.0);       // ARGUMENTO: 10.0
10 }
```

Na linha 2, `radius` é o *parâmetro* — um placeholder que ainda não tem valor. Na linha 8, `r` é o *argumento* — um valor concreto que *vai ser copiado* para `radius` no momento da chamada. Na linha 9, o argumento é o literal `10.0`.

Por que essa distinção importa?

A diferenciação vem da terminologia matemática formal. Em $f(x) = x^2$, x é o *parâmetro formal* da função f ; quando avaliamos $f(3) = 9$, o 3 é o *argumento atual*. Programadores experientes podem usar os dois termos intercambiavelmente em conversação informal — mas, em código cuidadosamente documentado e em textos técnicos, preserva-se a diferença.

Observação de Engenharia de Software

A finalidade do parâmetro é fornecer à função a *informação adicional* de que ela precisa para realizar sua tarefa. Sem parâmetros, uma função só pode operar sobre dados que estejam visíveis em seu escopo (variáveis globais ou dados-membro). Parâmetros tornam funções *reutilizáveis*: a mesma função `areaCirculo` serve para qualquer raio.

Síntese conceitual

Os quatro exercícios de autorrevisão do Capítulo 16 cobrem o *vocabulário básico* da orientação a objetos em C++ (16.1), as *convenções sintáticas* (16.2), a distinção crucial entre *variável*

local e *dado-membro* (16.3) e entre *parâmetro* e *argumento* (16.4). Esse vocabulário será aplicado, em escala prática, nos sete exercícios de programação do capítulo — onde escreveremos, pela primeira vez na obra, classes completas: `GradeBook`, `Account`, `Fatura`, `Empregado`, `Date`, `FrequenciaCardiaca` e `PerfilSaude`.
